Simulador de pedidos con planificación Round Robin y sincronización por semáforos

Proceso, planificación y sincronización aplicados a un problema productor-consumidor

Antecedentes

La planificación de procesos y la sincronización entre hilos son dos de los temas centrales en sistemas operativos. Casi todo sistema que atiende varias tareas al mismo tiempo, ya sea un servidor web, una base de datos o una aplicación con varios hilos, necesita resolver dos preguntas: quién usa el procesador en cada momento y cómo evitar que dos tareas se pisen al acceder al mismo recurso.

El problema productor-consumidor es el ejemplo más usado para enseñar sincronización, porque combina un recurso compartido acotado, el buffer, con dos tipos de proceso que compiten por él. Round Robin, por su parte, es uno de los algoritmos de planificación más simples de implementar y entender, y sirve como base antes de pasar a esquemas con prioridades o tiempos estimados.

Para este trabajo se decidió no quedarse en la versión abstracta del problema, con procesos genéricos que producen y consumen datos, sino llevarlo a un caso concreto: un sistema de pedidos donde los clientes piden productos y los trabajadores los preparan contra un inventario. También se pidió que la solución quedara en JavaScript, con una arquitectura que más adelante se pueda separar en cliente-servidor o en microservicios.

Planteamiento del problema

El reto tenía tres partes. Primero, programar uno o varios procesos con etapas bien definidas y puntos donde tenga sentido pausar y reanudar su ejecución. Segundo, programar un planificador Round Robin que reparta el procesador entre esos procesos de forma justa, con quantum configurable y cambios de contexto visibles. Tercero, programar un algoritmo de sincronización, en este caso semáforos, que impida condiciones de carrera sobre los recursos compartidos.

A eso se sumó una restricción práctica: si solo se sincroniza el acceso a una cola de datos genérica, es difícil ver para qué sirve todo esto fuera del salón de clases. Por eso se buscó un escenario donde la falta de sincronización produjera un error reconocible: la sobreventa de inventario, un problema real y frecuente en sistemas de comercio electrónico cuando varios procesos leen y modifican el mismo stock sin control.

Solución

La solución modela un sistema de pedidos con dos tipos de proceso: clientes, que arman pedidos de producto y cantidad y los colocan en una cola compartida, y trabajadores, que retiran pedidos de la cola y los despachan contra el inventario.

La cola de pedidos se sincroniza con tres semáforos clásicos, mutex, empty y full, igual que en el problema productor-consumidor de manual. El inventario se protege con un cuarto semáforo, invMutex, porque es un recurso distinto al de la cola y tiene su propia condición de carrera: si dos trabajadores leen el stock disponible antes de que ninguno lo actualice, ambos pueden despachar el mismo producto y dejar el inventario en negativo.

Semáforo	Valor inicial	Qué protege
mutex	1	Exclusión mutua sobre la cola de pedidos (solo un proceso a la vez la lee o la escribe).
empty	tamaño del buffer	Espacios libres en la cola; evita que un cliente escriba en una cola llena.
full	0	Pedidos disponibles; evita que un trabajador lea una cola vacía.
invMutex	1	Exclusión mutua sobre el inventario, en la operación de verificar y descontar stock.

Sobre esa base se implementó un planificador Round Robin que decide, en cada turno, qué proceso recibe el procesador y por cuántas instrucciones. Todo el sistema corre en un servidor Node.js y se expone por WebSocket, de modo que varios navegadores pueden conectarse a la vez, ver el estado de la simulación en tiempo real y enviar sus propios pedidos, que compiten por el mismo inventario que los procesos automáticos.

Implementación

El proyecto quedó organizado en dominio y transporte, separados a propósito:

- process.js define los procesos como generadores de JavaScript. Cada yield representa una instrucción atómica, como esperar un semáforo o escribir en el buffer, lo que permite pausar un proceso en cualquier punto y retomarlo después, tal como lo haría un sistema operativo real con un hilo.
- scheduler.js implementa el Round Robin: mantiene una cola de listos, ejecuta hasta un quantum de instrucciones por turno, y mueve el proceso al final de la cola si se le acaba el quantum, o lo saca de la cola si se bloquea en un semáforo.
- semaphore.js implementa el semáforo contador con wait y signal, y su propia cola de bloqueados, sin depender de promesas, de modo que es el planificador, y no el orden de ejecución de JavaScript, quien decide cuándo se reanuda un proceso.
- buffer.js modela la cola de pedidos acotada.
- inventory.js modela el stock por producto y la operación de verificar y descontar, que es la que queda protegida por invMutex.
- simulation.js conecta todas las piezas anteriores sin conocer nada de HTTP ni de WebSocket.
- server.js levanta un servidor Node con el módulo http y la librería ws, y transmite el estado de la simulación a todos los clientes conectados en cada turno del planificador.
- index.html y client.js forman la interfaz web: muestran la cola de pedidos, el inventario, el valor de cada semáforo, quién está bloqueado en cada uno, y la bitácora de eventos del planificador. Incluyen además un formulario para que un usuario real envíe un pedido.

La simulación se probó en el navegador: se agregaron clientes y trabajadores, se cambió el quantum y la velocidad, y se envió un pedido manual con una cantidad mayor a la disponible para verificar que el sistema lo rechazara en lugar de dejar el inventario en negativo.

El proceso o hilo simulado

Cada cliente o trabajador es una instancia de SimProcess, construida sobre un generador de JavaScript. El método step() avanza el generador una instrucción, y feedResult() le entrega el resultado de la instrucción anterior, para que expresiones como const order = yield {...} reciban el valor real que produjo esa instrucción.

process.js — clase SimProcess

```
class SimProcess {
  constructor(type, generatorFn) {
    this.id = `${type[0].toUpperCase()}${nextId++}`;
    this.type = type;
    this.iterator = generatorFn(this.id);
    this.state = STATES.READY;
    this.blockedOn = null;
    this._pendingInput = undefined;
  }

  feedResult(value) {
    this._pendingInput = value;
  }

  step() {
    const { value, done } = this.iterator.next(this._pendingInput);
    this._pendingInput = undefined;
    if (done) {
      this.state = STATES.DONE;
      return null;
    }
    return value;
  }
}
```

Las instrucciones de cada hilo

producerProcess y consumerProcess son generadores que describen, paso a paso, lo que hace un cliente o un trabajador. Cada yield es una instrucción que el planificador ejecuta una por una, lo que permite interrumpir al proceso entre cualquier par de instrucciones.

process.js — generador producerProcess (cliente)

```
function* producerProcess(bufferSize, id) {
  let seq = 0;
  while (true) {
    const product = randomProduct();
    const qty = randomQty();
    yield { type: 'work', label: `${id} arma pedido: ${qty}x ${product}` };
    yield { type: 'sem_wait', sem: 'empty' };
    yield { type: 'sem_wait', sem: 'mutex' };
    yield { type: 'buffer_put', item: { orderId: `${id}-${seq}`, client: id, product, qty
} };
    yield { type: 'sem_signal', sem: 'mutex' };
    yield { type: 'sem_signal', sem: 'full' };
    seq++;
  }
}
```

process.js — generador consumerProcess (trabajador)

```
function* consumerProcess(id) {
  while (true) {
    yield { type: 'sem_wait', sem: 'full' };
    yield { type: 'sem_wait', sem: 'mutex' };
    const order = yield { type: 'buffer_take' };
    yield { type: 'sem_signal', sem: 'mutex' };
    yield { type: 'sem_signal', sem: 'empty' };

    yield { type: 'sem_wait', sem: 'invMutex' };
    yield { type: 'inventory_process', order };
    yield { type: 'sem_signal', sem: 'invMutex' };

    yield { type: 'work', label: `${id} termina de gestionar pedido` };
  }
}
```

Bloqueo y activación de hilos en el semáforo

tryWait() decide si un proceso puede continuar o debe bloquearse; signal() libera el recurso y devuelve, si corresponde, el proceso que estaba esperando para que el planificador lo reactive.

semaphore.js — tryWait y signal

```
tryWait(process) {
    this.value -= 1;
    if (this.value >= 0) return true;
    this.blocked.push(process);
    return false;
  }

  signal() {
    this.value += 1;
    if (this.value <= 0) {
      return this.blocked.shift();
    }
    return null;
  }
```

En el planificador, block() saca al proceso de la cola de listos y lo marca como bloqueado, mientras que wake() lo regresa a la cola cuando otro proceso ejecuta signal() sobre el semáforo correspondiente.

scheduler.js — block y wake

```
wake(process) {
    process.state = STATES.READY;
    process.blockedOn = null;
    this.readyQueue.push(process);
  }

  block(process, semName) {
    process.state = STATES.BLOCKED;
    process.blockedOn = semName;
  }
```

Planificación Round Robin

runTurn() toma el proceso al frente de la cola de listos y le asigna la CPU por un número fijo de instrucciones (el quantum). Si el proceso agota su quantum sin bloquearse ni terminar, vuelve al final de la cola; si se bloquea en un semáforo, sale de la cola hasta que otro proceso lo despierte.

scheduler.js — runTurn

```
runTurn(executeInstruction) {
    if (this.readyQueue.length === 0) return null;

    const process = this.readyQueue.shift();
    process.state = STATES.RUNNING;

    let ticksUsed = 0;
    let outcome = 'quantum_expired';

    while (ticksUsed < this.quantum) {
      const instruction = process.step();
      ticksUsed++;

      if (instruction === null) {
        this.finish(process);
        outcome = 'finished';
        break;
      }

      const result = executeInstruction(process, instruction);
      process.feedResult(result && result.value);

      if (result && result.blocked) {
        outcome = 'blocked';
        break;
      }
    }

    if (outcome === 'quantum_expired' && process.state === STATES.RUNNING) {
      process.state = STATES.READY;
      this.readyQueue.push(process);
    }

    return { processId: process.id, ticksUsed, outcome };
  }
```

Resultados

Con la simulación en marcha se observó lo siguiente:

- La cola de pedidos nunca superó su capacidad ni se leyó estando vacía, lo que confirma que empty, full y mutex funcionan como se esperaba.

- La bitácora mostró los cambios de contexto del Round Robin: cada proceso recibió su quantum, y los procesos bloqueados en un semáforo salieron de la cola de listos hasta que otro proceso los liberó con signal.
- El inventario bajó de forma consistente incluso con varios trabajadores activos a la vez, sin quedar en negativo en ningún momento.
- Al enviar un pedido manual de 50 manzanas, con un stock disponible menor, el pedido se rechazó y el contador de rechazados por falta de stock aumentó, mientras que el inventario se mantuvo en un valor válido.
- Varias pestañas del navegador conectadas al mismo servidor mostraron el mismo estado en tiempo real, lo que confirma que la arquitectura cliente-servidor funciona con múltiples clientes simultáneos.

Conclusiones

El ejercicio permitió llevar tres conceptos de teoría, proceso, planificación y sincronización, a una implementación que se puede ejecutar, observar y modificar. Usar generadores de JavaScript para representar procesos resultó práctico porque permite pausar la ejecución en cualquier instrucción sin recurrir a hilos reales del sistema operativo, lo que facilita programar el planificador desde cero.

El caso del inventario compartido mostró que la sincronización importa más allá del ejercicio de manual: sin ella, un sistema real puede vender más de lo que tiene en existencia, y ese error es fácil de pasar por alto si el diseño no considera las condiciones de carrera desde el principio.

Separar el dominio, procesos, planificador, semáforos e inventario, de la capa de transporte deja el proyecto listo para crecer. Agregar otro algoritmo de planificación, como prioridades o el más corto primero, implicaría cambiar solo el archivo del planificador, y mover el inventario o la cola de pedidos a un servicio aparte no requeriría tocar la lógica de sincronización que ya existe.